

Spatial Atlas: Compute-Grounded Reasoning for Spatial-Aware Research Agent Benchmarks

Typed evidence and metric perception support traceable computation. Stateful action-loop integration remains the next step.

Arun Sharma | Advisor: Prof. Shashi Shekhar

Department of Computer Science and Engineering, University of Minnesota, Twin Cities

1. Overview

Objective: Compute deterministic subproblems before language generation (CGR).

Limit: Answer-only scores cannot distinguish grounded geometry from guessing.

KEY TAKEAWAY

Computing what can be computed before generating what must be generated.

2. Spatial Atlas System

Two benchmark-oriented handlers share one A2A server, model routing, budgets, and provenance controls.

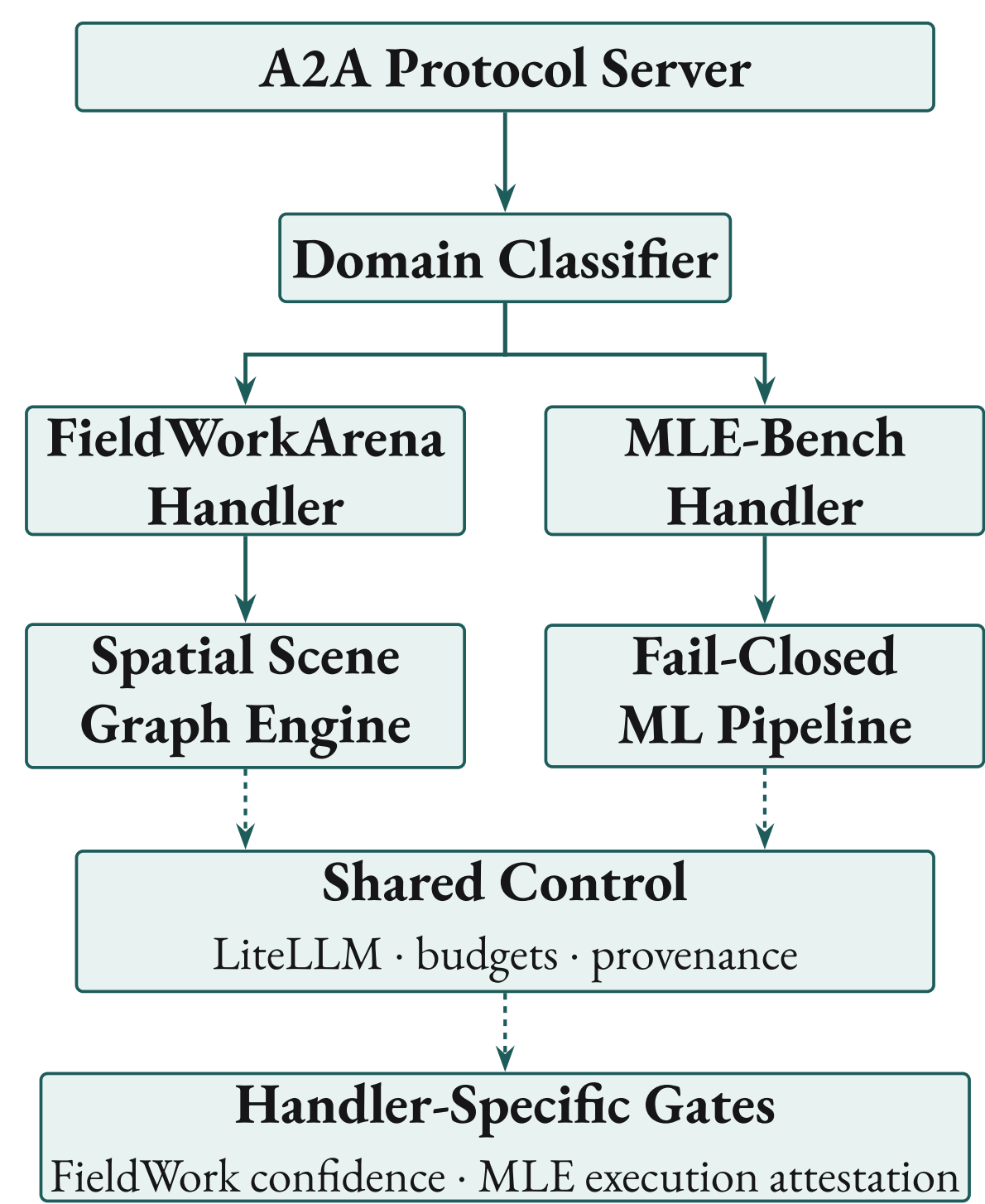


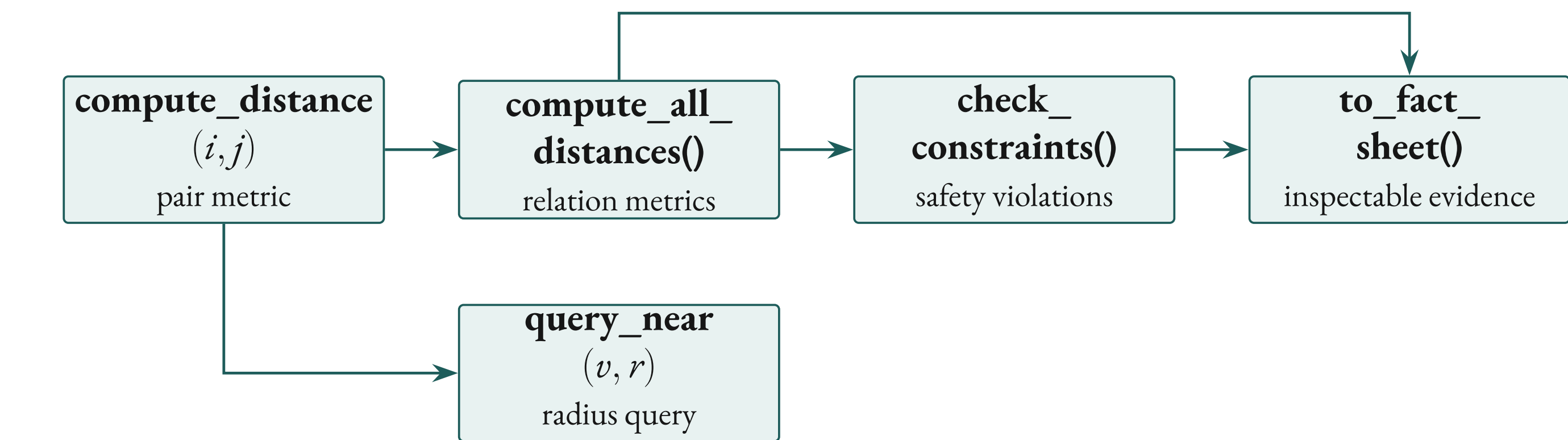
Figure 1. Spatial Atlas architecture. A classifier routes tasks to two handlers that use shared control services and apply handler-specific gates. Dashed lines indicate shared services.

3. Contributions

Computable evidence substrate: Given an image I and question q , CGR separates evidence construction from answer generation by creating a typed graph $G = (V, E)$:

$v_i = \text{SpatialEntity}(\text{id}_i, \ell_i, \text{pos}_i, \text{attrs}_i, z_i)$ (1) $e_{ij} = \text{SpatialRelation}(i, p_{ij}, j, d_{ij})$ (2)

Deterministic graph program: The graph exposes four computation and query methods plus one fact-sheet serializer over vision-language-estimated coordinates, producing structured evidence rather than measured 3D geometry.



SCENE-GRAPH DATAFLOW DAG

- **Scene-graph flow:** Extraction proposes entities and approximate positions $\rightarrow$ structuring writes typed vertices and relations $\rightarrow$ computation evaluates geometry and constraints on the fixed graph. $\text{compute_distance}(i, j)$ feeds the bulk path $\text{compute_all_distances}() \rightarrow \text{check_constraints}() \rightarrow \text{to_fact_sheet}()$, which serializes stored distances and violations. It also feeds the independent read-only $\text{query_near}(v, r)$ radius branch, which is not required for serialization.
- **Entropy-Guided Reasoning:** At step t , the knowledge state $\mathcal{K}_t$ aggregates observations, computed facts, and intermediate conclusions. The paper defines answer entropy and selects the action with the greatest expected entropy reduction:

$$H(\mathcal{A} \mid \mathcal{K}_t) = - \sum_{a \in \mathcal{A}} P(a \mid \mathcal{K}_t) \log P(a \mid \mathcal{K}_t) \quad (3)$$

$$c^* = \arg \max_g \mathbb{E}[H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \text{obs}(c_g))] \quad (4) \quad \text{reflect}(a) = \begin{cases} \text{True}, & \sigma(a) < \tau \\ \text{False}, & \text{otherwise} \end{cases} \quad (5)$$

Approach and why it will succeed: Four mechanisms instantiate compute-grounded reasoning (CGR) end to end.

- **Typed scene evidence:** entities, estimated positions, relations, zones, and constraints.
- **Confidence-gated reflection:** one conditional refinement when self-graded confidence is below 0.6.
- **Fail-closed execution:** generated code requires explicit opt-in and isolated-worker attestation.
- **Repair then score-gated refinement:** diagnose and patch failures using error and stdout traces. After successful execution, retain a refinement only when its parsed validation score improves.

The spatial branch computes relations and facts. The ML branch analyzes tasks, generates and repairs pipelines, and compares pipeline-emitted validation scores.

Require: task T , state $\mathcal{K}_0$, budget B , threshold τ

```

1  $a_0, \sigma_0 \leftarrow \text{FastModel}(T, \mathcal{K}_0)$ 
2 if  $\sigma_0 \geq 0.8$  then
3   return  $a_0$ 
4 end if
5  $\mathcal{K}_1 \leftarrow \mathcal{K}_0 \cup \text{SceneGraph}(T)$ 
6  $a_1, \sigma_1 \leftarrow \text{StandardModel}(T, \mathcal{K}_1)$ 
7 for  $r = 1$  to 2 do
8   if  $\sigma_1 \geq \tau$  then
9     return  $a_1$ 
10  end if
11   $\mathcal{K}_{r+1} \leftarrow \text{Reflect}(\mathcal{K}_r, a_1)$ 
12   $a_1, \sigma_1 \leftarrow \text{StrongModel}(T, \mathcal{K}_{r+1})$ 
13 end for
14 return  $a_1$ 

```

4. Evaluation

FieldWorkArena Handler: The v2 preprint describes a pipeline using Florence-2, GPT-4.1 vision, and bounding-box-centroid positions. The current repository instead obtains `position_x` and `position_y` from a Strong-model JSON extraction prompt.

- 1 Describe the image and extract entities and relations.
- 2 Store a typed scene graph $G = (V, E)$.
- 3 Compute pairwise distances and safety-rule violations.
- 4 Pass a fact sheet to the final language model.

MLE-Bench Handler: Execution of generated code follows a fail-closed policy that requires explicit operator opt-in and isolated-worker attestation.

- 1 Analyze task type, metric, files, and constraints.
- 2 Generate a strategy-specific Python pipeline.
- 3 Run only inside the bounded worker after the gate.
- 4 Diagnose, patch, and retry failures for at most 3 attempts.
- 5 Parse VALIDATION_SCORE after success.
- 6 Refine at most twice and retain only a score improvement.

The rollback rule is explicit:

$$p_{t+1} = \begin{cases} p' & \text{better}(s(p'), s(p_t)) \\ p_t & \text{otherwise} \end{cases} \quad (6)$$

Prompted safeguards: The generated pipeline is instructed to check ID overlap, row fingerprints, temporal order, and byte-identical media. These checks are not independently enforced. Dummy fallback remains opt-in.

5. Post-Berkeley Operational Validation

Private V37 Label-Free Four-Arm Smoke			
Arm	Evidence path exercised	Rows	Status
Scene graph	Correct image with typed scene evidence	8	Complete
Question only	Question text without scene evidence	8	Complete
Metric	Correct image with mask and point-map geometry	8	Complete
Shuffled metric	Schema-matched shuffled-image control path	8	Complete
Total	Label-free prediction journals	32	Complete

Execution closure: One content-pinned root ran with zero retries. The root and all bounded service steps completed successfully. All eight bounded logs passed the required warning scan.

Teardown closure: Registry restoration passed, the job-local credential was absent after teardown, and the terminal finalizer created the smoke seal and terminal result.

SCIENTIFIC BOUNDARY

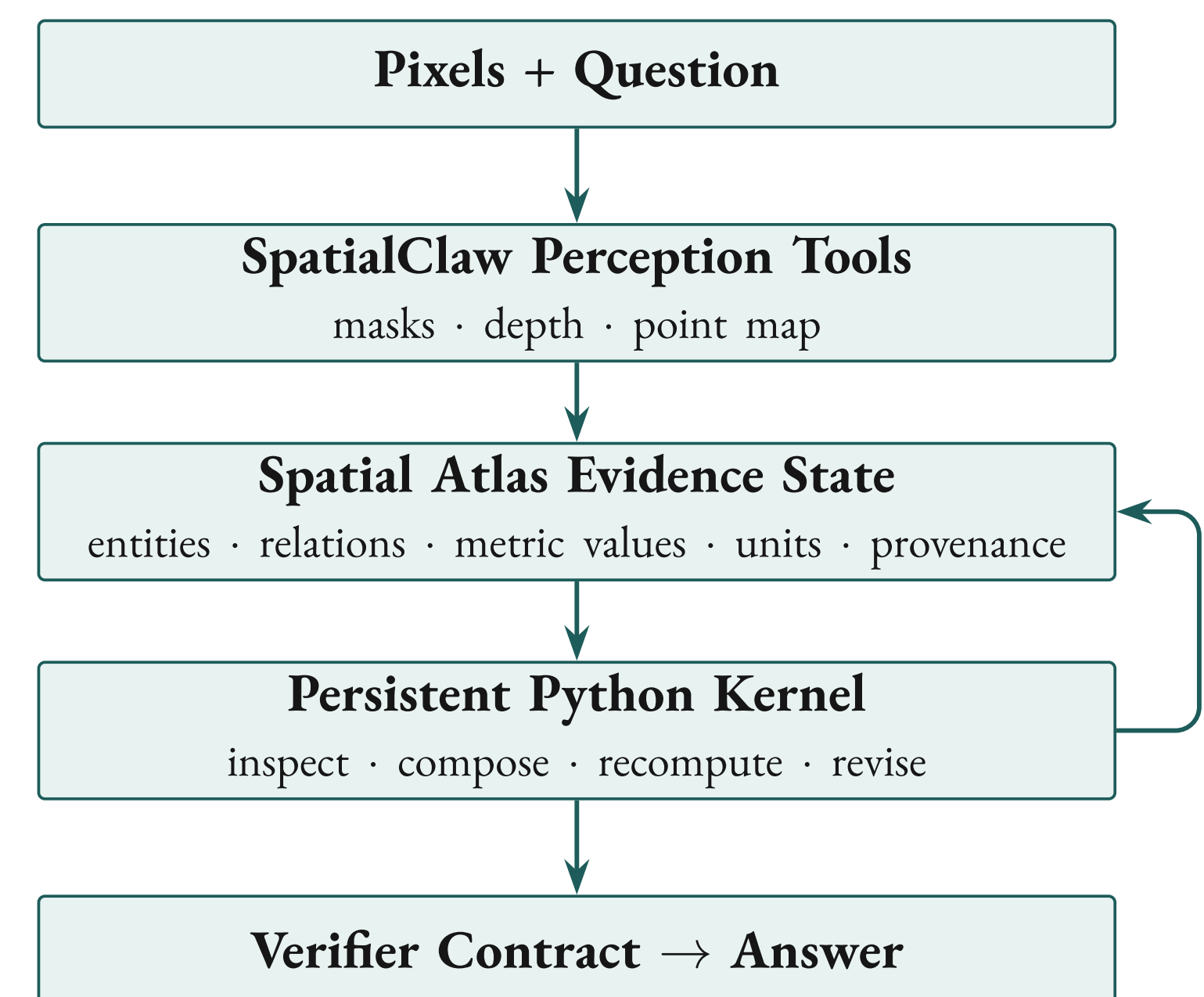
Protected labels and ground truth stayed sealed. No scoring or paired analysis occurred. V37 therefore establishes **operational integrity**, not benchmark accuracy or superiority.

Next: Protected scoring, paired analysis, and native persistent-loop integration remain future scientific work.



6. SpatialClaw Integration

Integration design: Spatial Atlas specifies the evidence that should be available before generation. SpatialClaw contributes a stateful code-action interface that can inspect tool outputs, compose operations, and revise computation after each observation.



Integration rationale: Four mechanisms from NVIDIA's SpatialClaw would enable stateful CGR.

- **Code as action:** AST-checked cells compose tools one step at a time.
- **Persistent state:** variables and outputs persist across steps.
- **Metric perception:** SAM3 and Depth-Anything-3 expose masks and reconstructed geometry.
- **Feedback-driven revision:** stdout, variables, and images inform the next step before `ReturnAnswer`.

Figure 2. Integration design. Private V37 exercised the public metric path in a label-free four-arm smoke. The persistent-kernel link, evidence-use journal, and post-kernel answer verifier remain proposed.

UPSTREAM CONTEXT

NVIDIA's SpatialClaw (Cho et al., 2026) is a separate, training-free framework that treats code as the action interface. A VLM writes one AST-checked Python cell per step in a persistent kernel, then conditions later actions on text, variables, errors, and rendered evidence. The kernel exposes SAM3 and Depth-Anything-3 perception primitives with scientific libraries. The SpatialClaw paper reports **59.9% average accuracy across 20 spatial reasoning benchmarks** and **+11.2 points** over the prior best spatial agent. These are SpatialClaw results, not Spatial Atlas evaluations. Its tasks, metrics, models, and perception stack differ.

7. Integration Contributions

7.1 Metric Perception Bridge (public)

The FieldWork handler can switch from the scene graph to a metric backend that invokes external SpatialClaw perception primitives. Strict QSpatial mode fails closed rather than falling back. External GPU services, gated data, and the frozen control artifact are not bundled.

- 1 Obtain object masks and a reconstructed point map.
- 2 Erode masks and retain finite, confidence-qualified points.
- 3 Build deterministic horizontal XZ voxel representatives.
- 4 Compute symmetric directed nearest-neighbor 5th percentiles.
- 5 Validate metric range, provenance, protocol contracts, and evidence hashes.

$$d_{A \rightarrow B} = Q_{0.5} \left(\left\{ \min_{b \in B} \|a - b\|_{XZ} : a \in A \right\} \right) \quad (7)$$

$$\hat{d}(A, B) = \min(d_{A \rightarrow B}, d_{B \rightarrow A}) \quad (8)$$

SpatialClaw supplies the perception primitives. Erosion, voxelization, the estimator, strict validation, the scene bridge, and the four run modes with their validators are current public Spatial Atlas source.

7.2 Target Action Contracts

The remaining persistent-loop integration requires:

- **Kernel-visible state:** expose entities, values, confidence, units, and provenance.
- **Stateful contract:** inspect, compose, recompute, and revise evidence per observation.
- **Evidence-use journal:** record facts and tool outputs used by each answer.
- **Verifier contract:** count only answers backed by fresh, valid, attested evidence.

TAKE-AWAY

The metric bridge and four-arm execution path are operationally exercised. Accuracy, the persistent action loop, and answer-level evidence-use verification remain unmeasured.